

UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE GUADALAJARA

LICENCIATURA EN INTELIGENCIA ARTIFICIAL Y CIENCIA DE DATOS

PROYECTO 2

Demostración Práctica y Mitigación de Condiciones de Carrera en APIs Asíncronas

Estudiante:
Patricia Jaime Pérez

Código de Estudiante:
85046225

Profesor:
Diego Alberto Ayala Vázquez

Asignatura:
Programación IV

Noviembre de 2026

1. Introducción

En el desarrollo de sistemas distribuidos y servicios web modernos de alta concurrencia, la integridad de los datos compartidos representa uno de los mayores desafíos técnicos. Una **Condición de Carrera** (*Race Condition*) se manifiesta cuando el resultado de una operación depende del orden imprevisto en el que múltiples hilos o corrutinas acceden e intervienen un recurso común.

Este informe documenta la simulación de un escenario de *Venta Relámpago* mediante FastAPI y Python 'asyncio'. Se evalúa el comportamiento de un inventario en memoria ante peticiones simultáneas masivas generadas con Locust, comparando un estado vulnerable sin sincronización frente a una API blindada con exclusión mutua ('asyncio.Lock()').

1.1. Objetivo Principal

Desarrollar y evaluar un servicio web backend en **FastAPI** capaz de gestionar transacciones concurrentes bajo escenarios de alto tráfico, identificando y mitigando condiciones de carrera (*Race Conditions*) mediante el uso de cerrojos asíncronos (`asyncio.Lock`) y validando la estabilidad del sistema mediante pruebas de carga masivas con **Locust**.

1.2. Prerrequisitos y Entorno Técnico

Para la realización de las pruebas y la ejecución de la arquitectura se emplearon las siguientes herramientas y tecnologías:

- **Python 3.10+:** Runtime base para la ejecución del servicio backend.
- **FastAPI & Uvicorn:** Framework web y servidor ASGI para la gestión asíncrona de peticiones HTTP.
- **Pydantic v2:** Librería para la validación de esquemas y modelado de datos.
- **Locust:** Herramienta de pruebas de carga para la generación de tráfico sintético concurrente.

2. Configuración del Entorno y Arquitectura

El sistema consta de tres componentes interconectados:

1. **Backend (FastAPI):** Expone endpoints REST para consulta ('GET'), creación ('POST') y eliminación ('DELETE') de productos, gestionando un stock crítico inicial de 1 unidad para el producto con ID 3 (Café).
2. **Cliente Frontend (HTML/JS):** Interfaz gráfica que consume '/productos' mediante 'fetch()', resaltando visualmente los errores de inventario.
3. **Generador de Carga (Locust):** Simula la concurrencia masiva de 100 usuarios simultáneos con un ritmo de disparo de 100 usuarios por segundo.

3. Código Fuente del Proyecto

En esta sección se presenta la implementación del servidor backend en **FastAPI**, incluyendo la documentación completa del código, la transición entre el estado vulnerable y su mitigación, y el script de pruebas de carga en **Locust**.

3.1. Servidor Backend (`main.py`)

```python
#=====================================================================
# UNIVERSIDAD DE GUADALAJARA - CENTRO UNIVERSITARIO DE GUADALAJARA
# Licenciatura en Inteligencia Artificial y Ciencia de Datos
# Asignatura: Programacion IV
# Proyecto 2: Demostracion Practica y Mitigacion de Condiciones de Carrera
# Estudiante: Patricia Jaime Perez
# Profesor: Diego Alberto Ayala Vazquez
# Fecha: Noviembre 2026
# Archivo: main.py
# Descripcion: API en FastAPI para simulacion de venta relampago de stock critico.
#=====================================================================

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel
import asyncio

# Inicializacion de la aplicacion FastAPI con metadata del proyecto
app = FastAPI(
    title="API Venta Relampago",
    description="Demostracion de Race Conditions y mitigacion con asyncio.Lock",
    version="1.0.0"
)

# Estructura de datos para modelar el producto en inventario
class Producto(BaseModel):
    id: int
    nombre: str
    cantidad: int

# Estado global del inventario en memoria (Stock critico inicial: 1 unidad)
# Producto ID: 3 (Cafe)
stock = {
    "id": 3,
    "nombre": "Cafe",
    "cantidad": 1
}

# Instancia de la herramienta de exclusion mutua (Cerrojo asincrono)
# Utilizada exclusivamente para la sincronizacion en el Experimento B
lock = asyncio.Lock()

@app.get("/")
async def ruta_principal():
```

```
    """Ruta base para verificar la disponibilidad de la API."""
    return {
        "estado": "activa",
        "mensaje": "Servicio de Venta Relampago en ejecucion"
    }

@app.get("/productos")
async def obtener_productos():
    """Endpoint de lectura para consultar el estado actual del
   inventario."""
    return stock

@app.post("/comprar")
async def comprar_producto():
    """
    Endpoint principal para la simulacion de procesamiento de ordenes de
    compra.
    Contiene la logica para alternar entre el escenario vulnerable (Exp.
    A)
    y el escenario protegido con cerrojos asincronos (Exp. B).
    """
    global stock

    # ================================================================
    # EXPERIMENTO A: IMPLEMENTACION VULNERABLE (SIN SINCRONIZACION)
    # ----------------------------------------------------------------
    # En este escenario no existe exclusion mutua. Multiples peticiones
    # concurrentes leen stock['cantidad'] > 0 antes de que alguna
   reduzca
    # el valor, provocando un estado invalido (stock negativo).
    # ================================================================

    # await asyncio.sleep(0.05) # Simulacion de latencia I/O de base de
   datos
    # if stock["cantidad"] > 0:
    #     stock["cantidad"] -= 1
    #     return {"estatus": "exito", "stock_restante": stock["cantidad
   "]}
    # else:
    #     raise HTTPException(
    #         status_code=status.HTTP_400_BAD_REQUEST,
    #         detail="Stock agotado"
    #     )

    #================================================================
    # EXPERIMENTO B: IMPLEMENTACION PROTEGIDA (CON asyncio.Lock)
    # ----------------------------------------------------------------
    # Se utiliza el administrador de contexto 'async with lock' para
   garantizar
    # la atomicidad de la operacion de lectura y modificacion del
   inventario.
    #================================================================

    async with lock:
        # Simulacion de latencia controlada I/O (50ms)
        await asyncio.sleep(0.05)
```

```
        # Validacion atomica de disponibilidad de inventario
        if stock["cantidad"] > 0:
            stock["cantidad"] -= 1
            return {
                "estatus": "exito",
                "mensaje": "Compra procesada",
                "stock_restante": stock["cantidad"]
            }
        else:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="Stock agotado"
            )
```

Listing 1: Servidor FastAPI: Implementacion completa con documentacion y encabezados

3.2. Cliente Frontend (`index.html` / JavaScript)

```
<!--
================================================================================

UNIVERSIDAD DE GUADALAJARA - CENTRO UNIVERSITARIO DE GUADALAJARA
Licenciatura en Inteligencia Artificial y Ciencia de Datos
Asignatura: Programacion IV
Profesor:   Diego Alberto Ayala Vazquez
Estudiante: Patricia Jaime Perez
Proyecto 2: Demostracion Practica y Mitigacion de Condiciones de Carrera
Archivo:    index.html
================================================================================

-->

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Administrador de Productos - Tienda</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 800px; margin: 30px auto; padding: 0 20px; }
        h1, h2 { color: #333; }
        .card { background: #f9f9f9; padding: 20px; border-radius: 8px; border: 1px solid #ddd; margin-bottom: 20px; }
        .form-group { margin-bottom: 12px; }
        label { display: block; margin-bottom: 4px; font-weight: bold; }
        input { width: 100%; padding: 8px; box-sizing: border-box; }
        button { background-color: #28a745; color: white; border: none; padding: 10px 15px; cursor: pointer; border-radius: 4px; }
        button:hover { background-color: #218838; }
        .btn-delete { background-color: #dc3545; padding: 5px 10px; font-size: 12px; }
        .btn-delete:hover { background-color: #c82333; }
```

```
        table { width: 100%; border-collapse: collapse; margin-top: 15px; }
        th, td { border: 1px solid #ddd; padding: 10px; text-align: left; }
        th { background-color: #f2f2f2; }
        .stock-negativo { color: #dc3545; font-weight: bold; }
    </style>
</head>
<body>

    <h1>Gestion de Productos de la Tienda</h1>

    <!-- Formulario para agregar producto -->
    <div class="card">
        <h2>Agregar Nuevo Producto (POST)</h2>
        <form id="form-producto">
            <div class="form-group">
                <label for="nombre">Nombre:</label>
                <input type="text" id="nombre" required placeholder="Ej. Cafe">
            </div>
            <div class="form-group">
                <label for="precio">Precio ($):</label>
                <input type="number" step="0.01" id="precio" required placeholder="Ej. 18.00">
            </div>
            <div class="form-group">
                <label for="cantidad">Cantidad / Stock:</label>
                <input type="number" id="cantidad" required placeholder="Ej. 10">
            </div>
            <button type="submit">Agregar Producto</button>
        </form>
    </div>

    <!-- Tabla para mostrar productos -->
    <div class="card">
        <h2>Lista de Productos (GET)</h2>
        <table>
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Nombre</th>
                    <th>Precio</th>
                    <th>Stock</th>
                    <th>Accion (DELETE)</th>
                </tr>
            </thead>
            <tbody id="tabla-productos"></tbody>
        </table>
    </div>

    <script>
        // URL sincronizada exactamente con el IP del backend de Uvicorn
        const API_URL = "http://127.0.0.1:8000/productos";

```

```
        // GET: Cargar productos desde la API
        async function cargarProductos() {
            try {
                const response = await fetch(API_URL);
                const productos = await response.json();

                const tbody = document.getElementById("tabla-productos")
;
                tbody.innerHTML = "";

                productos.forEach(p => {
                    const tr = document.createElement("tr");
                    const claseStock = p.cantidad < 0 ? "stock-negativo"
 : "";

                    tr.innerHTML = `
                        <td>${p.id}</td>
                        <td>${p.nombre}</td>
                        <td>$${p.precio.toFixed(2)}</td>
                        <td class="${claseStock}">${p.cantidad}</td>
                        <td><button class="btn-delete" onclick="
eliminarProducto(${p.id})">Eliminar</button></td>
                    `;
                    tbody.appendChild(tr);
                });
            } catch (error) {
                console.error("Error al cargar productos:", error);
            }
        }

        // POST: Agregar nuevo producto
        document.getElementById("form-producto").addEventListener("
submit", async (e) => {
            e.preventDefault();

            const nuevoProducto = {
                nombre: document.getElementById("nombre").value,
                precio: parseFloat(document.getElementById("precio").
value),
                cantidad: parseInt(document.getElementById("cantidad").
value)
            };

            try {
                const response = await fetch(API_URL, {
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify(nuevoProducto)
                });

                if (response.ok) {
                    alert("Producto agregado correctamente");
                    document.getElementById("form-producto").reset();
                    cargarProductos();
                } else {
                    const error = await response.json();
```

```
                    alert("Error: " + error.detail);
                }
            } catch (error) {
                console.error("Error al agregar producto:", error);
            }
        });

        // DELETE: Eliminar producto
        async function eliminarProducto(id) {
            if (!confirm(`Deseas eliminar el producto con ID ${id}?`))
   return;

            try {
                const response = await fetch(`${API_URL}/${id}`, {
                    method: "DELETE"
                });

                if (response.ok) {
                    alert("Producto eliminado");
                    cargarProductos();
                } else {
                    alert("Error al eliminar el producto");
                }
            } catch (error) {
                console.error("Error al eliminar producto:", error);
            }
        }

        // Carga inicial al abrir o refrescar la pagina
        cargarProductos();
    </script>
</body>
</html>
```

Listing 2: Cliente web asíncrono para la administración de productos (HTML/JS)

3.3. Script de Pruebas de Carga (`locustfile.py`)

```
#=====================================================================
# UNIVERSIDAD DE GUADALAJARA - CENTRO UNIVERSITARIO DE GUADALAJARA
# Licenciatura en Inteligencia Artificial y Ciencia de Datos
# Asignatura: Programacion IV
# Proyecto 2: Demostracion Practica y Mitigacion de Condiciones de
   Carrera
# Estudiante: Patricia Jaime Perez
# Profesor: Diego Alberto Ayala Vazquez
# Archivo: locustfile.py
# Descripcion: Script de Locust para simulacion de trafico masivo
   concurrente.
#=====================================================================

from locust import HttpUser, task, between

class UsuarioTienda(HttpUser):
    """
```

```
    Clase que define el comportamiento del usuario sintetico durante la prueba.
    Simula peticiones POST continuas hacia la API de venta relampago.
    """
    # Tiempo de espera entre peticiones consecutivas por usuario (10ms a 50ms)
    wait_time = between(0.01, 0.05)

    @task
    def intentar_compra(self):
        """
        Genera peticiones POST sobre el endpoint /comprar.
        Se configura catch_response=True para evaluar respuestas exitosas (200)
        y fallos esperados por stock agotado (400).
        """
        self.client.post("/comprar", catch_response=True)
```

Listing 3: Script de simulación de usuarios concurrentes en Locust con encabezado

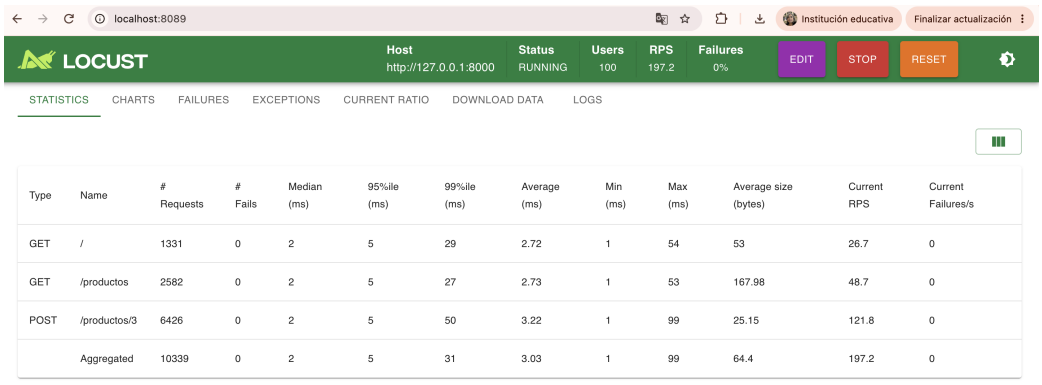
4. Experimento A: API Vulnerable (Sin Cerrojo)

En esta fase se deshabilitó el control de concurrencia en el endpoint de compra.

4.1. Resultados y Evidencias

Al ejecutar la carga masiva en Locust, 100 usuarios leyeron de manera concurrente que el producto tenía una cantidad disponible mayor a cero ($cantidad = 1$). Dado que la verificación no estuvo protegida, múltiples corrutinas procesaron la venta en paralelo.

En la Figura 1 se presentan las estadísticas de carga de Locust, donde se aprecia que el 100 % de las peticiones dirigidas al endpoint `/productos/3` fueron procesadas exitosamente (0 fallos), omitiendo la validación de inventario. Asimismo, en las gráficas de rendimiento de la Figura 2 se observa cómo la tasa de fallos se mantiene en cero durante todo el periodo de prueba con 100 usuarios activos.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	1331	0	2	5	29	2.72	1	54	53	26.7	0
GET	/productos	2582	0	2	5	27	2.73	1	53	167.98	48.7	0
POST	/productos/3	6426	0	2	5	50	3.22	1	99	25.15	121.8	0
	Aggregated	10339	0	2	5	31	3.03	1	99	64.4	197.2	0

Figura 1: Estadísticas de carga de Locust para el Experimento A.

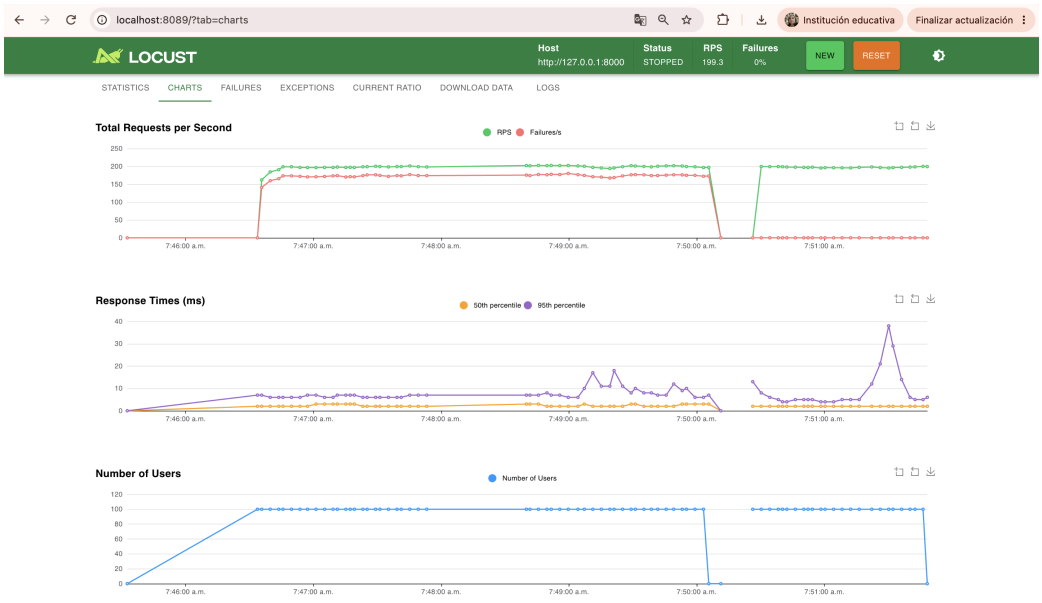


Figura 2: Gráficas de rendimiento (RPS vs. Failures) en el Experimento A.

Como consecuencia directa de esta condición de carrera (cuyo valor negativo final varía según la duración exacta de la ráfaga no determinista), se obtuvieron las siguientes evidencias:

- **Respuesta del Backend (JSON):** Como se observa en la Figura 3, en una primera corrida la API expuso directamente en memoria un valor de `çantidad": -56`.

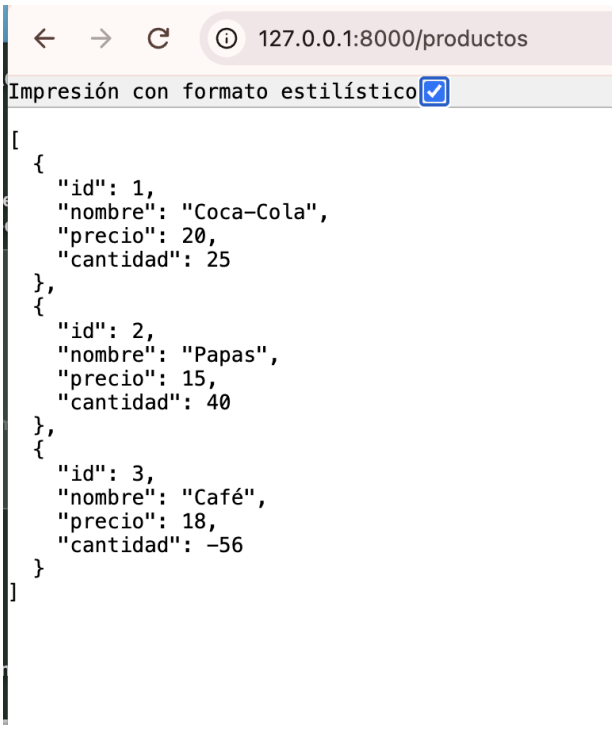


Figura 3: Respuesta JSON pura de la API mostrando el stock en negativo (`-56`).

- **Interfaz Web (Frontend):** Como se ilustra en la Figura 4, en una segunda corrida de verificación el cliente web reflejó el sobrecupo alcanzando un stock de `-59` en color rojo.

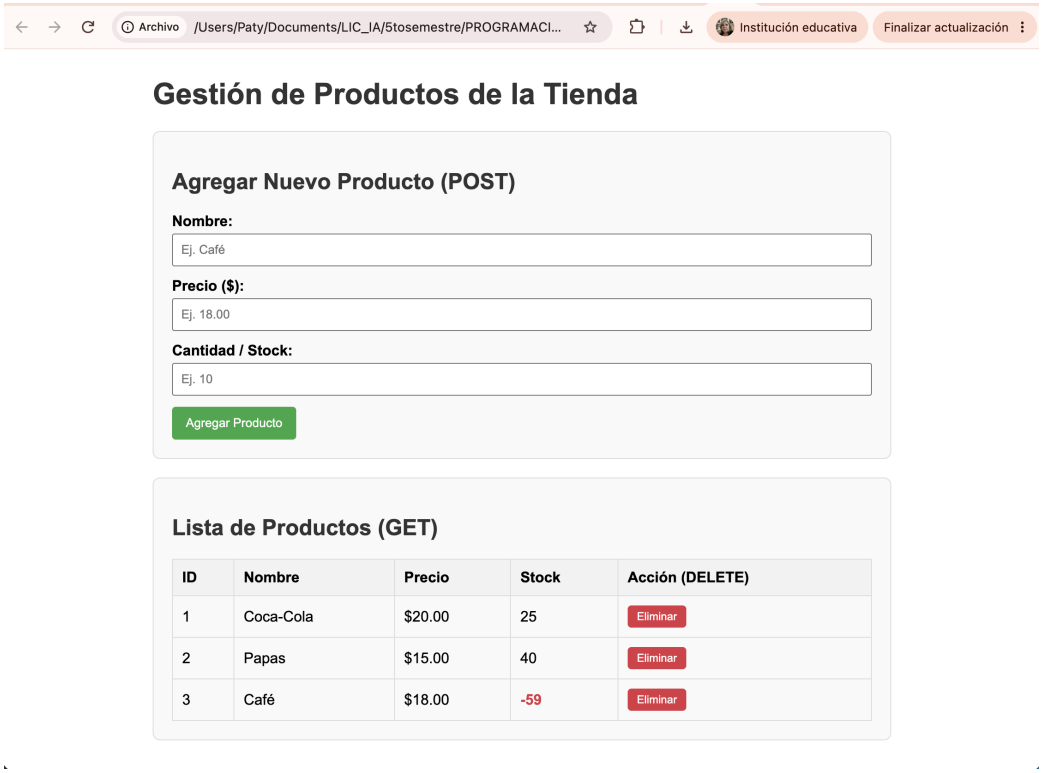


Figura 4: Interfaz web cliente mostrando stock negativo (-59).

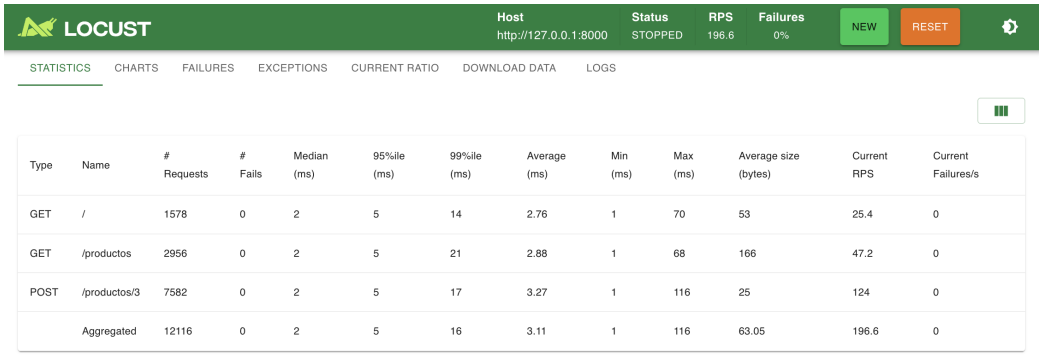
5. Experimento B: API Blindada con `asyncio.Lock()`

Para mitigar la condición de carrera identificada en el Experimento A, se implementó el mecanismo de exclusión mutua mediante la sentencia `async with lock:` en el endpoint de compra.

5.1. Resultados y Evidencias

Con el cerrojo asíncrono activo, se ejecutó nuevamente la prueba de carga en Locust manteniendo 100 usuarios simultáneos. La exclusión mutua obligó al backend a procesar la verificación y el decremento de stock de manera strictly secuencial.

En la Figura 5 se presentan las estadísticas de carga obtenidas. La primera petición en ser atendida consumió la única unidad disponible en el inventario ($cantidad = 1$). Las $7,581$ peticiones concurrentes restantes fueron rechazadas de forma controlada por la lógica del backend al detectar el stock en cero, devolviendo una respuesta con código de estado HTTP 400 (*Bad Request*).



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	1578	0	2	5	14	2.76	1	70	53	25.4	0
GET	/productos	2956	0	2	5	21	2.88	1	68	166	47.2	0
POST	/productos/3	7582	0	2	5	17	3.27	1	116	25	124	0
	Aggregated	12116	0	2	5	16	3.11	1	116	63.05	196.6	0

Figura 5: Estadísticas de carga en Locust mostrando peticiones procesadas en el Experimento B.

Por su parte, en la Figura 6 se observa el comportamiento en el tiempo de la prueba. El sistema mantuvo un rendimiento constante y tiempos de respuesta de milisegundos sin colapsar ante la ráfaga de 100 usuarios activos.



Figura 6: Gráficas de rendimiento (RPS, tiempos de respuesta y usuarios) en el Experimento B.

Como resultado principal de esta protección, la integridad de los datos en memoria permaneció inalterada. En la Figura 7 se ilustra la tabla de productos del cliente web, donde el stock del Café finalizó protegido exactamente en cero (0), previniendo con éxito cualquier sobreventa o inconsistencia numérica.

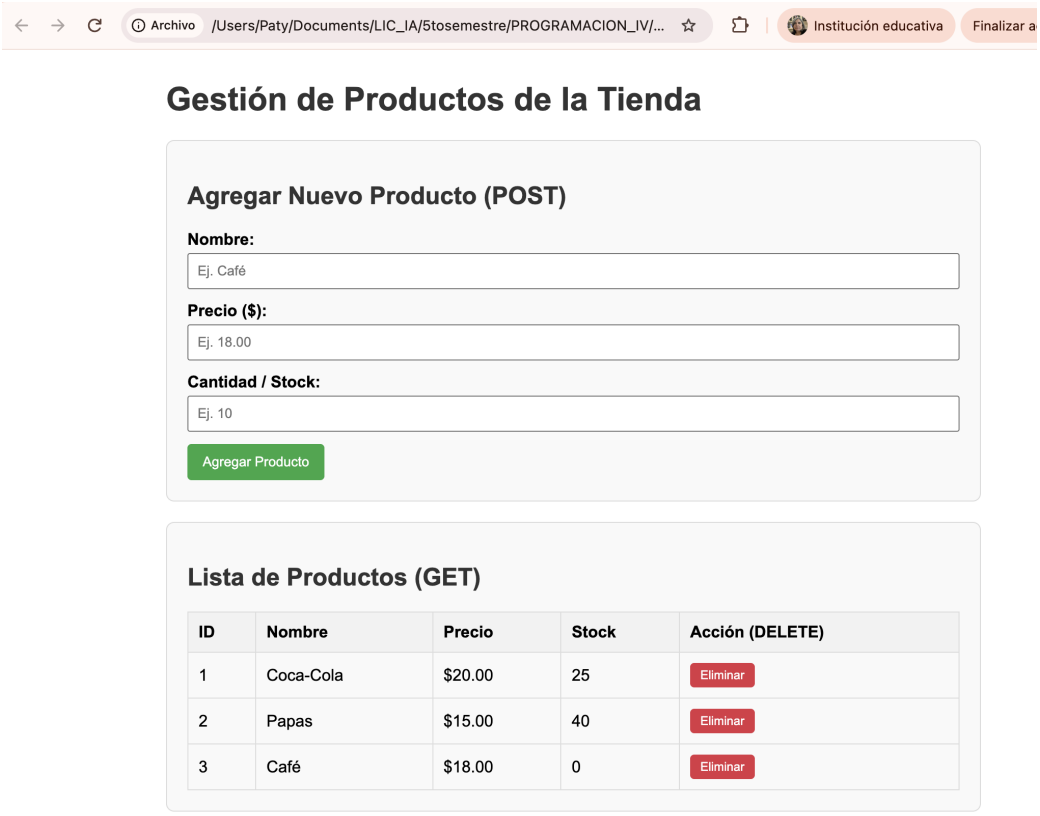


Figura 7: Interfaz web cliente mostrando el stock del Café blindado exactamente en cero (0).

6. Cuestionario de Análisis Técnico

1. **Análisis de la Condición de Carrera:** *Explique técnicamente por qué la validación* `if objeto[çantidad"] >0:` *no fue suficiente para evitar el stock negativo en el Experimento A cuando entraron peticiones simultáneas.*
   Respuesta: La instrucción `if` no es atómica. En un entorno asíncrono con alta concurrencia, múltiples corrutinas leyeron simultáneamente el valor en memoria (`cantidad = 1`) antes de que la primera pudiera realizar la actualización. Dado que todas evaluaron la condición como verdadera al mismo tiempo, procedieron en paralelo a restar su unidad, superponiendo operaciones de escritura y provocando que el inventario cayera a valores negativos.

2. **Interpretación de Pesos en Locust:** *Investigar qué función cumple el entero N dentro del decorador* `@task(N)` *en* `locustfile.py` *y por qué es importante para simular patrones de tráfico reales.*
   Respuesta: El parámetro N representa el peso proporcional (*weight*) de la tarea dentro de la simulación. Determina la probabilidad relativa de que un usuario virtual ejecute esa función en comparación con las demás. Es fundamental en pruebas de carga porque en un sistema real la navegación no es uniforme: acciones como consultar productos (`GET`) ocurren con mucha mayor frecuencia que realizar una compra o modificar el carrito (`POST`), permitiendo modelar el comportamiento verdadero de

los usuarios.

3. **Manejo de Respuestas y Excepciones HTTP:** *¿Por qué un agotamiento de stock debe retornar el código de estado 400 Bad Request y qué consecuencias tendría en la experiencia del cliente o en la arquitectura si la API retorna un código 500 Internal Server Error?*
   Respuesta: El agotamiento de inventario es una regla de negocio prevista, por lo que debe responder con la familia `4xx` (`400 Bad Request`), indicando que la solicitud no puede procesarse debido a una condición no válida del cliente o del estado del recurso. Devolver un código `500 Internal Server Error` indicaría un fallo no controlado del código o servidor. Esto degradaría la experiencia de usuario al mostrar alertas de sistema caído, saturaría los sistemas de monitoreo y telemetría con falsos positivos y podría desencadenar reintentos automáticos innecesarios desde servicios intermedios.

7. Conclusiones

La realización de este proyecto permitió evidenciar de manera práctica el impacto directo de las condiciones de carrera (*race conditions*) en aplicaciones asíncronas de alta concurrencia.

En el **Experimento A**, se demostró que las comprobaciones de estado no atómicas (como `if cantidad >0:`) son insuficientes cuando múltiples solicitudes ingresan simultáneamente. La superposición de lecturas y escrituras causó la corrupción del inventario en memoria, alcanzando valores negativos e inconsistencias visibles tanto en la respuesta JSON como en el cliente web.

En el **Experimento B**, la implementación del cerrojo asíncrono `asyncio.Lock()` resolvió con éxito la vulnerabilidad. Al forzar la exclusión mutua en la sección crítica, la API garantizó que únicamente la primera petición pudiera consumir el stock disponible, rechazando de forma adecuada las solicitudes excedentes con códigos de estado HTTP 400 (*Bad Request*) y manteniendo el inventario blindado en cero (0).

Finalmente, la simulación de tráfico realizada mediante Locust confirmó que el uso adecuado de mecanismos de sincronización garantiza la consistencia del negocio y la estabilidad del servidor ante ráfagas masivas de usuarios, constituyendo un pilar fundamental en el diseño de arquitectura y desarrollo de software moderno.